

SAE optimisation clustering



MILOIKOVITCH Maxime, MARCOLE Matteo, ADAM Tristan,
LOGEART Pierre

Algorithmes implémentés

KMEANS:

Entrées : K (nombre de clusters), Données

Initialisation :

Sélectionner aléatoirement K points comme centroïdes de départ.

Répéter jusqu'il n'y ait plus aucun changement) :

a. Affectation :

Pour chaque point, l'assigner au centroïde le plus proche.

b. Étape de mise à jour :

Pour chaque cluster, recalculer son centroïde.

Méthodes:

```
int[] cluster(double[][] descriptions);
double[][] initialiserCentroides(double[][] descriptions,
int nombreCaracteristiques);
boolean affecterObjets(double[][] descriptions, double[][]
centroides, int[] affectations);
void recalculerCentroides(double[][] descriptions,
double[][] centroides, int[] affectations, int
nombreCaracteristiques);
```

DBSCAN:

Entrées : Epsilon (rayon), MinPts (voisins minimum), Données

Pour chaque point non visité :

Marquer le point comme visité.

Chercher ses voisins (points à une distance $\leq$ Epsilon).

Si nombre de voisins $<$ MinPts :

Le mettre comme "Bruit".

Sinon (c'est un point cœur) :

Créer un nouveau Cluster avec ce point.

Étendre le cluster en inspectant ses voisins :

- Les ajouter au cluster actuel.

- S'ils ont eux-mêmes assez de voisins ($\geq$ MinPts), ajouter ces nouveaux voisins à l'inspection.

Méthodes:

```
int[] cluster(double[][] descriptions);
void expandCluster(int pointDepart, List<Integer> voisins,
int numeroCluster);
List<Integer> regionQuery(int point);
```

Problèmes traités et choix effectués

- La qualité de l'image influe sur le temps de traitement
- La bordure de l'image est compté comme un biome

Résolution de l'image	Nombre de pixels	Temps K-Means	Temps DBSCAN
100 x 100	10 000	10 ms	50 ms
500 x 500	250 000	250 ms	30 secondes
1000 x 1000	1 000 000	1 seconde	8 minutes
1920 x 1080	2 000 000	2 secondes	30 minutes
3840 x 2160	8 300 000	8 secondes	plusieurs heures

Résultats obtenus

Lancement avec paramètres personnalisés (8 biomes, flou par moyenne):
java app.Main Planete1.jpg 8 moyenne

Arguments: Chemin_Image, Nombre_Biomes, Type_Flou

Charger l'image depuis Chemin_Image.

Détection des Biomes avec K-Means:

biomes_detectes = DetecteurBiomes(image, Nombre_Biomes, Type_Flou)

Sauvegarder l'image floutée et la carte globale colorisée des biomes.

Détection des Écosystèmes avec DBSCAN:

Pour chaque biome :

Si le biome correspond au fond/contour -> L'ignorer.

Extraction du biome :

Récupérer sa couleur moyenne et son nom.

Générer et sauvegarder une image isolant uniquement ce biome.

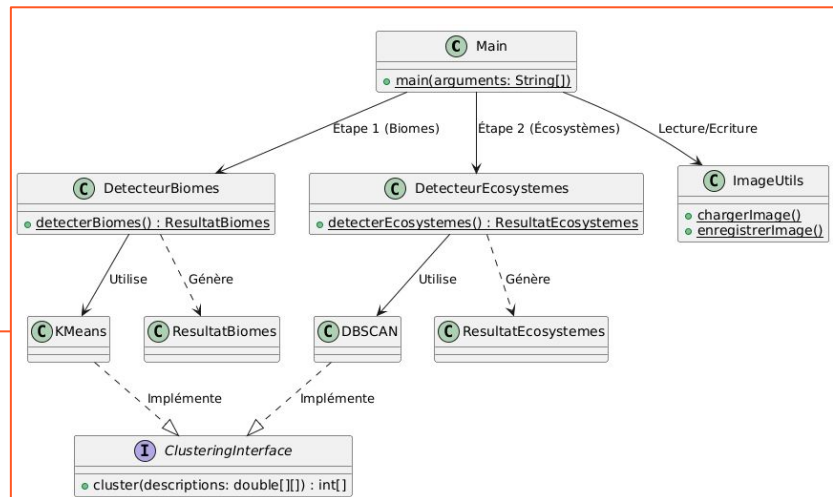
Découpage en écosystèmes :

ecosystemes = DetecteurEcosystemes(biomes_detectes, biome_actuel)

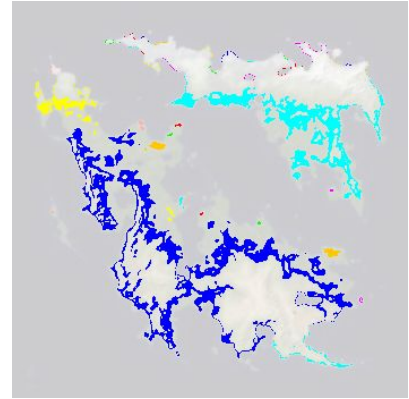
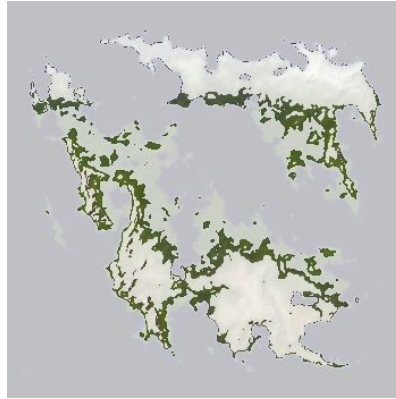
Générer et sauvegarder la carte des différents écosystèmes (zones connexes) pour ce biome.

Fin :

Résultats disponible en image.



Quelques images de résultats



Conclusion / perspectives

K-Means

- + Très rapide, parfait pour les grandes images.
- Le nombre de clusters (K) doit être connu, force des formes sphériques.

DBSCAN

- + Détecte des formes complexes, isole le bruit (anomalies), pas besoin de K.
- Très lent sur les hautes résolutions, sensible aux paramètres de densité.

Perspectives : Clustering Hiérarchique Ascendant (HAC)

- Construit un arbre de classification par fusions.
- Permet de définir le nombre de clusters après en coupant l'arbre.
- Une approche possible K-Means pour les grandes zones puis HAC sur les zones petites.